

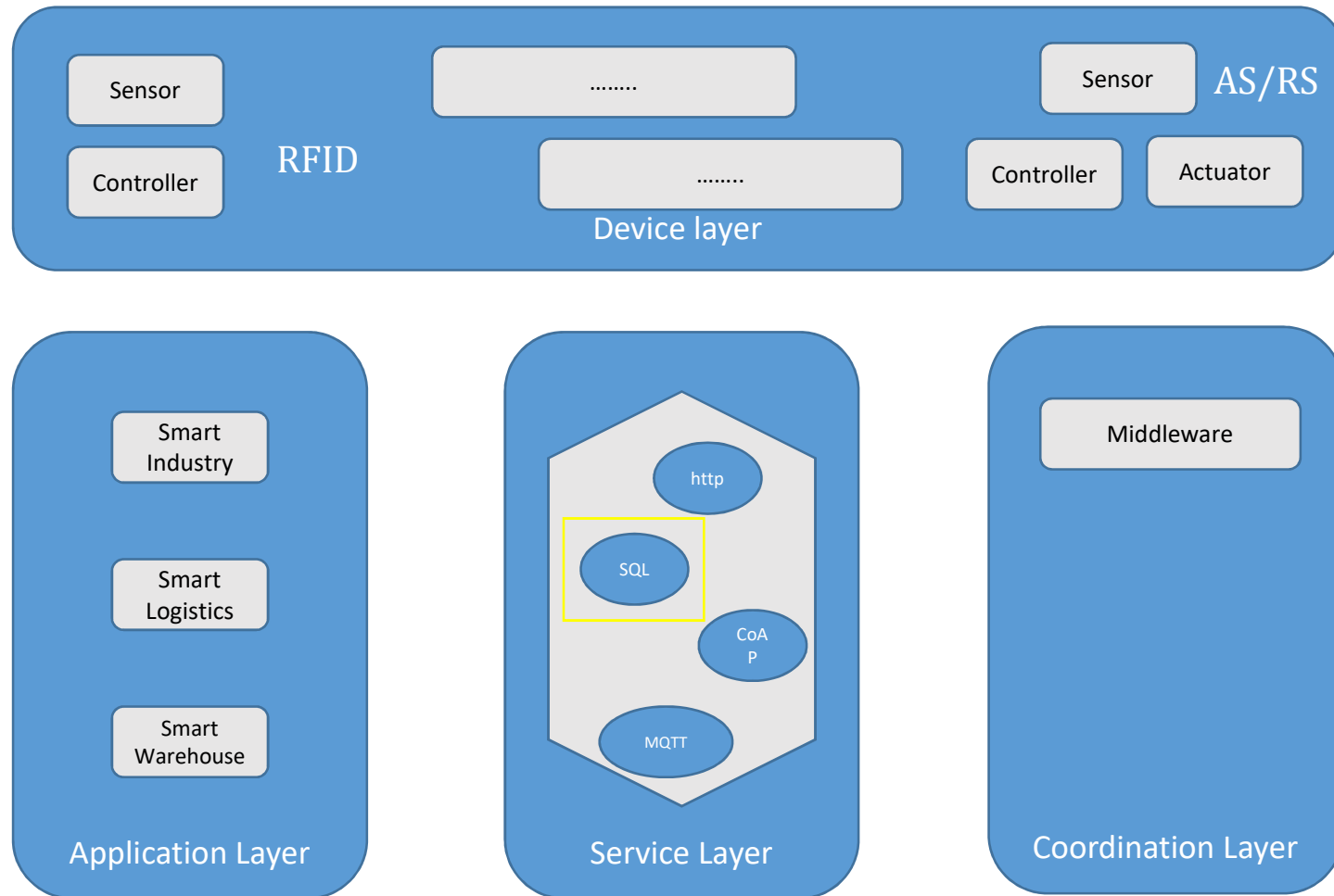
Advanced System Applications and Practice
for Internet and Multimedia
TM1117
Introduction to SQL

Instructor : Kelvin Leung

Industrial Centre
The Hong Kong Polytechnic University

An IoT Framework

Logistics automation systems



Introduction to SQL (4 hours Workshop)

After the training, you are able

- Understand how relational databases work
- Understand basic commands and functions of SQL
- Use SQL for data administration
- Use SQL commands for data manipulation
- Use SQL to query a database to extract useful information

What is Database?

- A database is an organized collection of data.
- Computer program can quickly select desired pieces of data.

customer_id	client_name	city	tel
1	Andrew	HONG KONG	12345678
2	Daniel	TOKYO	23456789
3	David	NEWYORK	34567890
4	John	TAIPEI	45678901
5	Noah	BEIJIN	56789012
6	William	SHANGHAI	67890123



```
Result
$sqlite3 database.sdb < main.sql
David
Andrew
```

What is a DBMS?

- A database management system is a collection of programs that provides management services for accessing a database.



What is a Relational DBMS

- RDBMS stores data in tables.
- Row is an entity and column provides information about that entity.

CUSTOMERS

<u>customer_id</u>	<u>client_name</u>	<u>city</u>	<u>tel</u>
1	Andrew	HONG KONG	12345678
2	Daniel	TOKYO	23456789
3	David	NEWYORK	34567890
4	John	TAIPEI	45678901
5	Noah	BEIJIN	56789012
6	William	SHANGHAI	67890123

Table name

Attribute name

WAYBILLS

<u>waybill_no</u>	<u>customer_id</u>	<u>tag_id</u>
1001	1	10000
1002	2	20000
1003	3	30000
1004	4	40000
1005	5	50000
1006	6	60000

Tuples or rows

What is SQL?

- Structured Query Language
- To interact with DBMS.
- Used to created and edit databases.
- Also used to create queries(P), forms(I), and reports(O) IPO model.

SQL History?

- IBM Sequel language developed as part of System R project at the IBM San Jose Research Laboratory
- Renamed Structured Query Language (SQL)
- ANSI and ISO standard SQL:

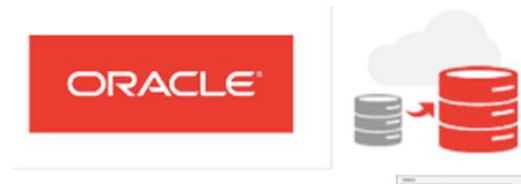
Year	Name	Also Known As	Changes
1986	SQL-86	SQL-87 (date when adopted by ISO)	First publication of the ANSI/ISO standard
1989	SQL-89		Only small revision of the original standard
1992	SQL-92	SQL2	Major update of the original standard and still the most widely supported standard
1999	SQL-99	SQL3	Update of the 1992 standard adding new ways of selecting data and new rules on data integrity and introducing object orientation
2003	SQL-2003		Introduced XML support and fields with autogenerated values

examples of DBMS applications

- Commercial software
 - Sybase Adaptive Server Enterprise
 - IBM DB2
 - Oracle
 - Microsoft SQL Server
 - Microsoft Access



- Free/GPL/Opensource:
 - MySQL
 - PostgreSQL



Database Design

- A good database design is, therefore, one that:
Divides your information into subject-based tables to reduce redundant data.
Provides Access with the information it requires to join the information in the tables together as needed.
Helps support and ensure the accuracy and integrity of your information.
Accommodates your data processing and reporting needs.

A good database design

- Divides the information into subject-based tables to reduce redundant data.
- Provides Access with the information it requires to join the information in the tables together as needed.
- Helps support and ensure the accuracy and integrity of the information.
- Accommodates the data processing and reporting needs.

Student_Info

SID	Name	Gender	HKID	Programme
S001	Tommy	M	H001	AscCS
S002	May	F	H002	AscCS
S003	Jack	M	H003	AOAL
S004	John	M	H004	AOAL
S005	Cathy	F	H005	AscIT
S006	Daisy	F	H006	AscIT

Note: (a) Primary key: SID

Student_Scores

SID	Course_code	Course_work_scores	Exam_scores	Exam_date
S001	C001	70	80.4	11 Jan. 2005
S001	C002	50	60.6	11 Feb. 2005
S001	C004	60	60.76	03 Jan. 2005
S002	C001	60	65.45	11 Jan. 2005
S002	C002	55	76.33	11 Feb. 2005
S002	C003	65	40.67	27 Feb. 2005

Note: (a) Primary key: SID + Course_code

Course_Info

Course_code	Course_work_scores	No_of_Credit	Exam_Fees	Lecturer
C001	Database	3	100	Dr. Lee
C002	Airport Management	3	150	Mr. Hung
C003	Logistics	2	120	Ms. Wong
C004	Information System	1	180	Dr. Lam
C005	Operation Research	4	130	Ms. Ching
C006	Airline Operations	3	200	Dr. Lee

Database design process

- 1) Determine the purpose of the database.
- 2) Find and organize the information required.
- 3) Divide the information into tables.
- 4) Turn information items into columns.
- 5) Analyze the design for errors.
- 6) Create the tables and add a few records of sample data. Make adjustments to the design, as needed.
- 7) Apply the normalization rules.

Syntax of Structured Query Language

- **Structured Query Language (SQL)** is the standard language for accessing information a database.
- SQL is case-insensitive and free format.
- SQL statements can use multiple lines
 - end each statement with a semi-colon ;

```
INSERT INTO CUSTOMERS VALUES(1,'Andrew', 'HONG  
KONG', '12345678' );
```

SQL statements end with **semi-colon**.

```
SELECT * FROM CUSTOMERS;
```

Some SQL Data types

SQLite Storage Classes

Sr.No.	Storage Class & Description
1	NULL The value is a NULL value.
2	INTEGER The value is a signed integer, stored in 1, 2, 3, 4, 6, or 8 bytes depending on the magnitude of the value.
3	REAL The value is a floating point value, stored as an 8-byte IEEE floating point number.
4	TEXT The value is a text string, stored using the database encoding (UTF-8, UTF-16BE or UTF-16LE)
5	BLOB The value is a blob of data, stored exactly as it was input.

Strings in SQL

Use **single quote mark** around String constants.

```
SELECT * FROM CUSTOMERS  
      WHERE name = 'Peter';
```

```
SELECT * FROM CUSTOMERS  
      WHERE district = 'Tue mun';
```

Table in SQL

- Database data is stored in tables
- Each table has a unique table name
- A table store a set of tuples, like a list

Table name

Attribute name

Tuples or rows

customer_id	client_name	city	tel
1	Andrew	HONG KONG	12345678
2	Daniel	TOKYO	23456789
3	David	NEWYORK	34567890
4	John	TAIPEI	45678901
5	Noah	BEIJIN	56789012
6	William	SHANGHAI	67890123

Table in SQL (cont'd)

- Columns have column name and data type
- Rows contain tuples for the columns
- Table constraints

The diagram illustrates the components of a SQL table. A box labeled 'Table name' points to the word 'CUSTOMERS'. A box labeled 'Attribute name' points to the 'tel' column header. A box labeled 'Tuples or rows' points to the first data row (1, Andrew, HONG KONG, 12345678).

<u>customer_id</u>	<u>client_name</u>	city	<u>tel</u>
1	Andrew	HONG KONG	12345678
2	Daniel	TOKYO	23456789
3	David	NEWYORK	34567890
4	John	TAIPEI	45678901
5	Noah	BEIJIN	56789012
6	William	SHANGHAI	67890123

Table Constraints - Primary key

- It is a column in a relation which take unique value for each row to identify the unique row(record).
- Primary key takes unique and not null characters.
- Primary key does not change over time.
- Primary keys will be very important to you as your knowledge of databases progresses.
- The primary key is used to relate (**join**) tables.

Table Primary key Example

- A table contains a *primary key* that uniquely identifies a row of data.

customer_id is the primary key in CUSTOMERS table.

CUSTOMERS



<u>customer_id</u>	<u>client_name</u>	city	<u>tel</u>
1	Andrew	HONG KONG	12345678
2	Daniel	TOKYO	23456789
3	David	NEWYORK	34567890
4	John	TAIPEI	45678901
5	Noah	BEIJIN	56789012
6	William	SHANGHAI	67890123

Creating Tables rules

- Table and column names must start with a letter
- They can not be key words of SQL command, e.g. create, insert etc.
- They can not exceed 30 characters.

Syntax:

```
create table "tablename"  
("columnname", "datatype",  
"columnname2", "datatype",  
"columnname3", "datatype");
```

Creating Tables

- Use “create table” command
- Syntax:

```
create table “tablename”  
  (“columnname”, “datatype”,  
   “columnname2”, “datatype”,  
   “columnname3”, “datatype”);
```

Creating Tables example

CUSTOMERS

<u>customer_id</u>	<u>client_name</u>	city	<u>tel</u>
1	Andrew	HONG KONG	12345678
2	Daniel	TOKYO	23456789
3	David	NEWYORK	34567890
4	John	TAIPEI	45678901
5	Noah	BEIJIN	56789012
6	William	SHANGHAI	67890123

Remarks



```
/* Create a table called CUSTOMERS*/  
CREATE TABLE CUSTOMERS(customer_id  
integer PRIMARY KEY, name text, city  
text, tel text);
```

Data Manipulation Commands

- Transaction
 - Adding table rows
 - Updating table rows
 - Deleting table rows
- Query
 - Listing table rows
 - Inserting table rows with a select subquery

Adding Table Rows

- INSERT

- Used to enter data into table

- Syntax:

- INSERT INTO *columnname*
VALUES (*value1*, *value2*, ... , *valuen*);

- e.g. Change customer name Andrew to Andy

INSERT INTO CUSTOMERS VALUES(1, 'Andrew', 'HONG KONG', '12345678');

Updating Table Rows

- update
 - Used to update or change records that match a specified criteria.
 - Syntax:
 - update "tablename" set "columnname" = "newvalue"
["nextcolumn" = "newvalue2"...]
where "columnname" OPERATOR "value"
[and|or "column" OPERATOR "value"];
 - e.g. Change customer_id=1 record's "customer name" "Andrew" to "Andy".
- UPDATE** CUSTOMERS **SET** name = 'Andy' **WHERE**
customer_id = 1;

Deleting Table Rows

- delete
 - Used to delete existing records in a table.
 - Syntax:
 - DELETE FROM *table_name*
WHERE *condition*;
 - e.g. Delete name's "Andy" in table CUSTOMERS.

```
DELETE FROM CUSTOMERS WHERE name = 'Andy';
```

SQL Select Statement

- The SELECT statement is used to pull information from a table.
 - **SELECT**
 - List the columns (and expressions) that should be returned from the query
 - **FROM**
 - Indicate the table(s) or view(s) from which data will be obtained
 - **WHERE**
 - Indicate the conditions under which a row will be included in the result
 - **GROUP BY**
 - Indicate categorization of results
 - **HAVING**
 - Indicate the conditions under which a category (group) will be included
 - **ORDER BY**
 - Sorts the result according to specified criteria

Basic Query Structure

- A typical SQL query:

```
select a1, a2, ..., an (attribute)  
from   r1, r2, ..., rn (relation)  
where p          (predicate)
```

Example:

```
SELECT * FROM CUSTOMERS WHERE name = 'Andy';
```

The select Clause

- The **SELECT** clause lists the attributes desired in the result of a query.
- An asterisk(*) in the select clause denotes “all attributes”.
- The **WHERE** clause specifies conditions that the result must satisfy.

Example:

Find the names of all customers who name is “Andy”

```
SELECT * FROM CUSTOMERS WHERE name = 'Andy';
```

Logical operations

- OR

```
SELECT * FROM CUSTOMERS WHERE  
District='Causeway bay' OR District='North point';
```

- AND

```
SELECT NAME FROM CUSTOMERS WHERE  
DISTRICT = 'North point' AND PRICE > 100;
```

- NOT

```
SELECT * FROM CUSTOMERS WHERE  
NOT PRICE = 200;
```

Set operations

- IN

```
SELECT * FROM CUSTOMERS WHERE  
District IN ('Causeway bay', District='North point');
```

Functions in SQL

- Select can be used with **functions**:

SUM (expression)

MAX (expression)

MIN (expression)

COUNT (expression)

Example:

Count the number of record in table CUSTOMERS:

SELECT COUNT(*) FROM CUSTOMERS;

Select Example Using a Function

Find how many events with IN operation.

```
SELECT COUNT(*) FROM EVENTS  
WHERE operation = 'IN';
```

Note: with aggregate functions you can't have single-valued columns included in the SELECT clause

GROUP BY

- **GROUP BY** is used when you want to apply a function (count, sum, avg) to a group of rows having a common characteristic.
- **HAVING** is used to put a condition on the groups.

Example: Find the names and average product price of all customers whose average buying items price is greater than 200.

```
SELECT customer_id, AVG (price)
FROM CUSTOMERS
GROUP BY customer_id
HAVING avg (price) > 200;
```

Ordering the Display of Tuples

- List in alphabetic order the names of all instructors

```
SELECT * FROM CUSTOMERS
```

```
ORDER BY last_name ;
```

- We may specify **DESC** for descending order or **ASC** for ascending order, for each attribute; ascending order is the default.
 - Example **ORDER BY** *name* **DESC**
- Can sort on multiple attributes
 - Example: **ORDER BY** *last_name, name*

SQL Select Clause Examples

Display all column from table's "CUSTOMERS"

```
SELECT * FROM CUSTOMERS;
```

Display name and city columns from table's "CUSTOMERS"

```
SELECT name,city FROM CUSTOMERS;
```

Display name and city columns from table's "CUSTOMERS"
sort by city(in ascending order)

```
SELECT name,city FROM CUSTOMERS GROUP BY city;
```

SQL Select Examples (cont'd)

Display name and city columns from table's "CUSTOMERS"

```
SELECT name,city FROM CUSTOMERS;
```

Find events occurred after 15:00 from EVENTS table

```
SELECT timeEVENT FROM EVENTS WHERE timeEVENT > '15:00';
```

Subqueries

Use the result of one query as part of another query.

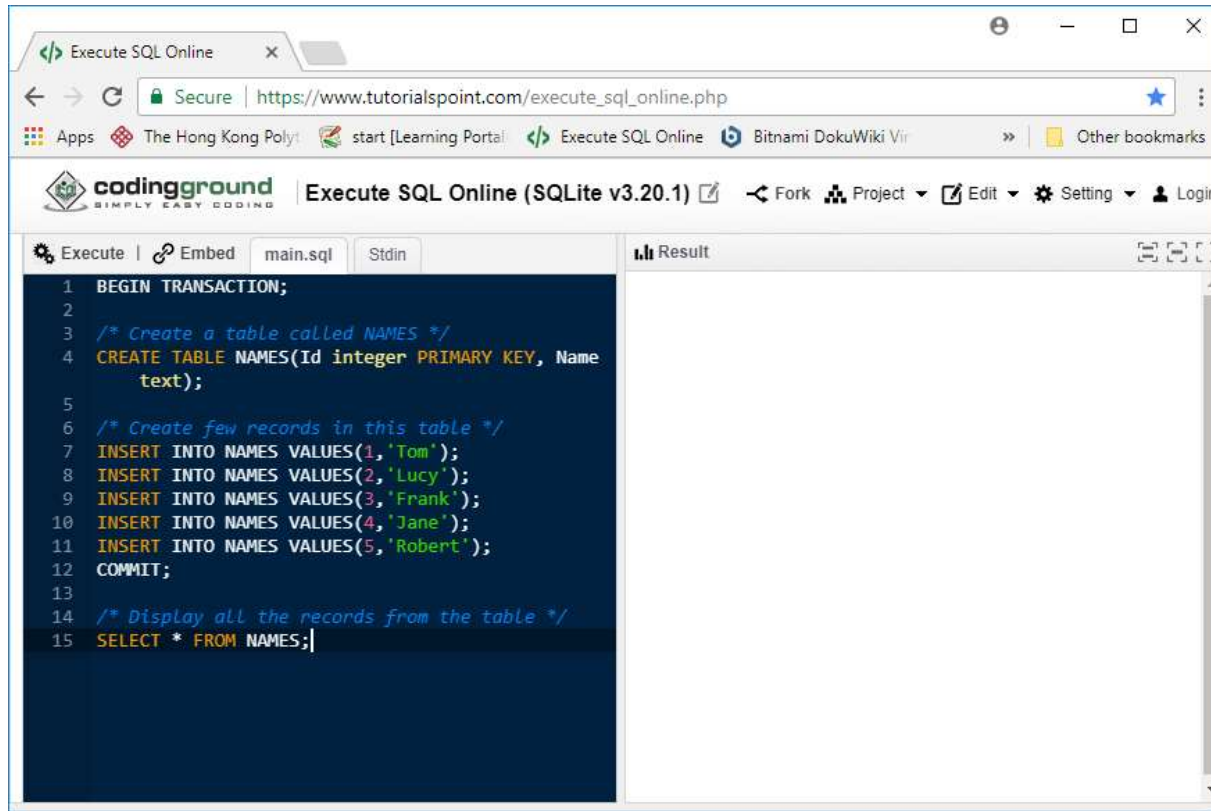
Example: Which customers has the highest price?

```
SELECT name,price
FROM CUSTOMERS
WHERE price =
(SELECT MAX (price) FROM CUSTOMERS);
```



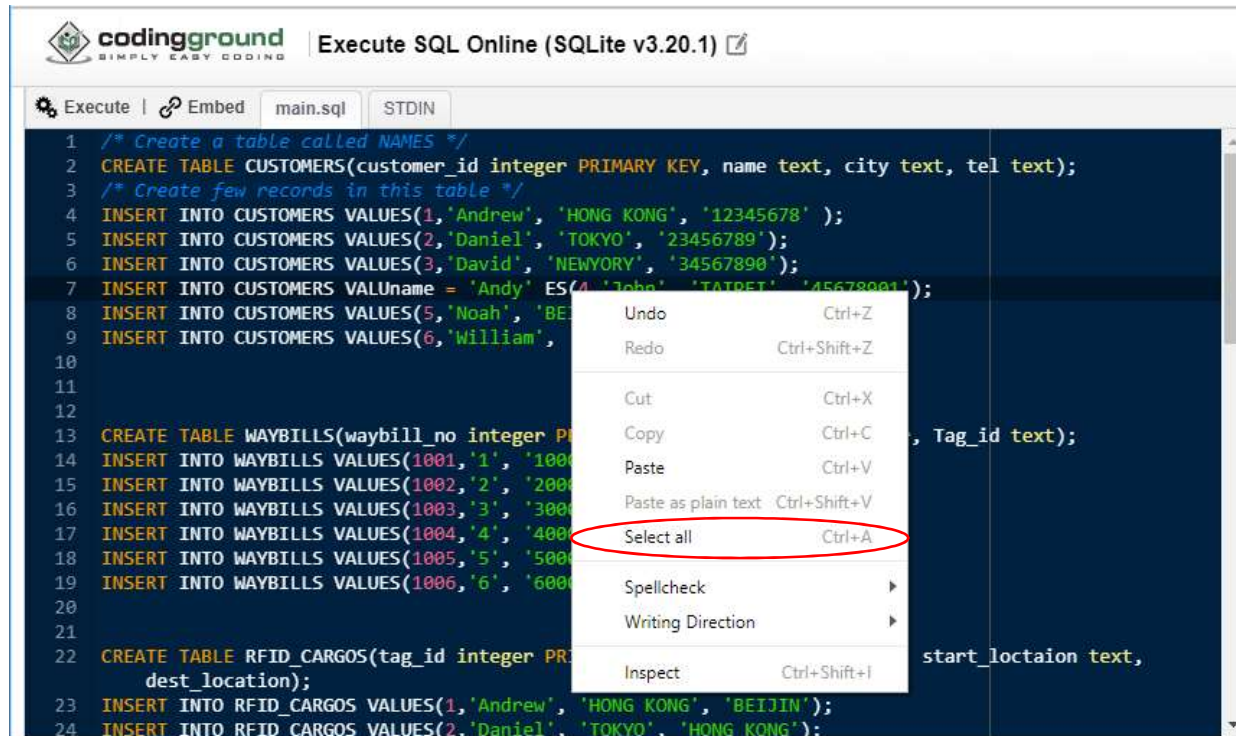
Subquery

Execute SQL Online



https://www.tutorialspoint.com/execute_sql_online.php

Prepare database for practices



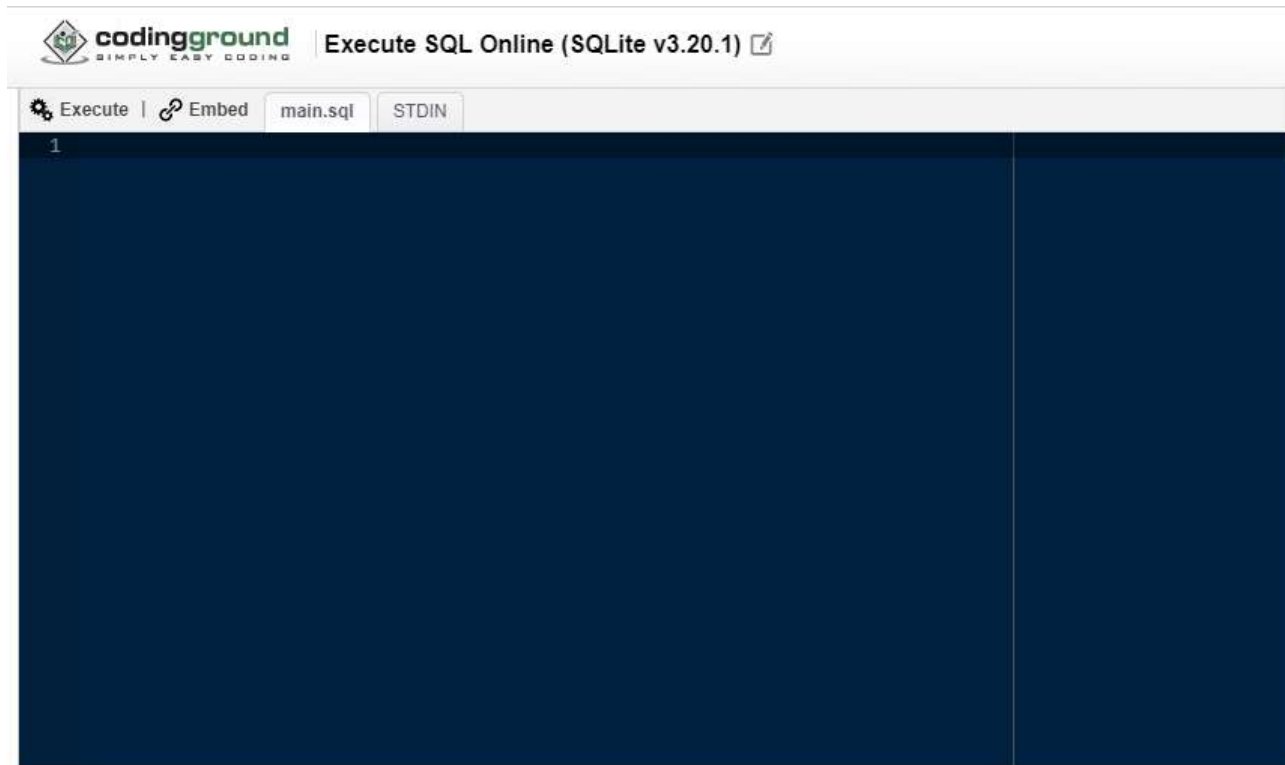
The screenshot shows the 'codingground' website's 'Execute SQL Online (SQLite v3.20.1)' interface. The main area contains a SQL script with the following content:

```
1  /* Create a table called NAMES */
2  CREATE TABLE CUSTOMERS(customer_id integer PRIMARY KEY, name text, city text, tel text);
3  /* Create few records in this table */
4  INSERT INTO CUSTOMERS VALUES(1,'Andrew', 'HONG KONG', '12345678' );
5  INSERT INTO CUSTOMERS VALUES(2,'Daniel', 'TOKYO', '23456789');
6  INSERT INTO CUSTOMERS VALUES(3,'David', 'NEWYORY', '34567890');
7  INSERT INTO CUSTOMERS VALUES(4,'John', 'TAIPEI', '45678901');
8  INSERT INTO CUSTOMERS VALUES(5,'Noah', 'BEIJING', '56789012');
9  INSERT INTO CUSTOMERS VALUES(6,'William', 'LONDON', '67890123');
10
11
12
13 CREATE TABLE WAYBILLS(waybill_no integer PRIMARY KEY, start_location text,
14 INSERT INTO WAYBILLS VALUES(1001,'1', '1000');
15 INSERT INTO WAYBILLS VALUES(1002,'2', '2000');
16 INSERT INTO WAYBILLS VALUES(1003,'3', '3000');
17 INSERT INTO WAYBILLS VALUES(1004,'4', '4000');
18 INSERT INTO WAYBILLS VALUES(1005,'5', '5000');
19 INSERT INTO WAYBILLS VALUES(1006,'6', '6000');
20
21
22 CREATE TABLE RFID_CARGOS(tag_id integer PRIMARY KEY, start_location text,
23 dest_location);
24 INSERT INTO RFID_CARGOS VALUES(1,'Andrew', 'HONG KONG', 'BEIJING');
25 INSERT INTO RFID_CARGOS VALUES(2,'Daniel', 'TOKYO', 'HONG KONG');
```

A context menu is open over the script, showing options: Undo (Ctrl+Z), Redo (Ctrl+Shift+Z), Cut (Ctrl+X), Copy (Ctrl+C), Paste (Ctrl+V), Paste as plain text (Ctrl+Shift+V), **Select all (Ctrl+A)** (highlighted with a red circle), Spellcheck, Writing Direction, and Inspect (Ctrl+Shift+I).

1. Right-click on main.sql screen then click “Select All or press “Ctrl-A” key.
2. Click “Delete” key to clear all previous data.

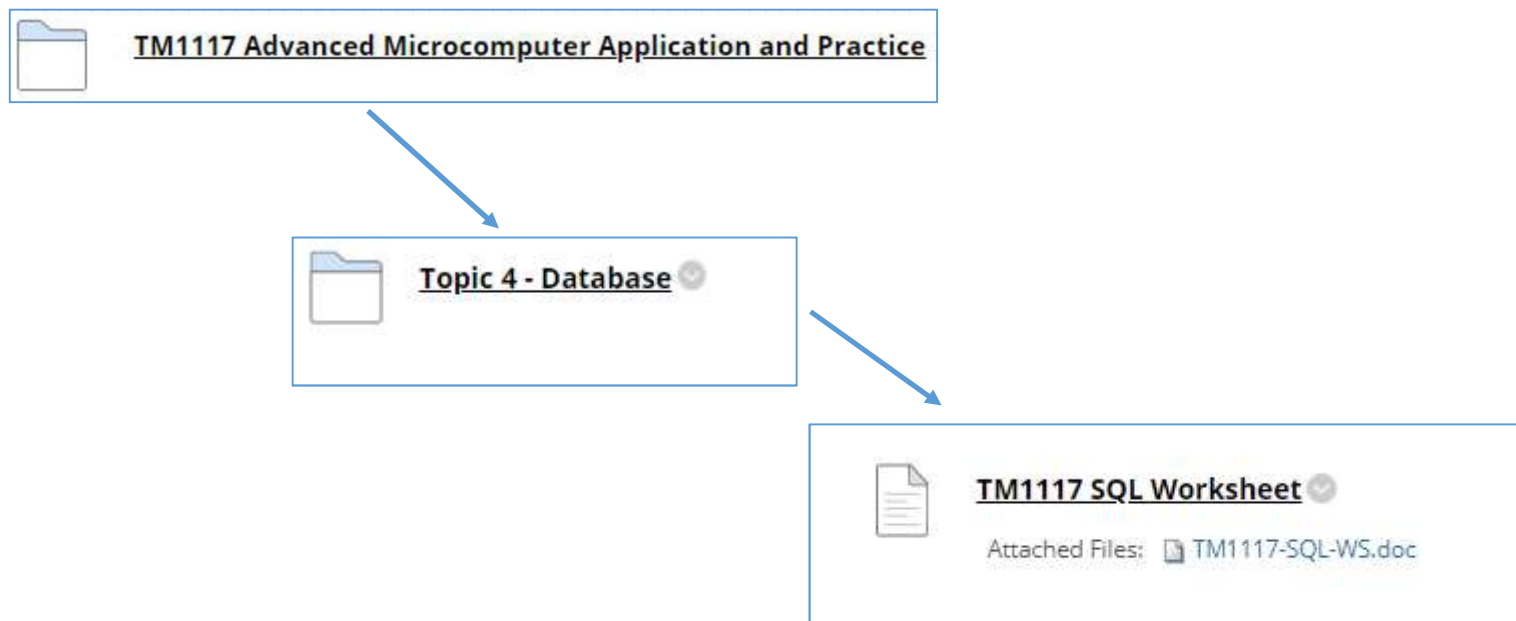
Clear all previous database



Click “Delete” key to clear all previous data.

Worksheet – Exercise 1:

- Complete the questions in the Blackboard
- Write SQL statements to answer the questions



Processing Multiple Tables

Previous exercises work with only one table

The most beneficial features of SQL and relational database systems is “Join” make relational database systems with “Relational”.

Entity data in the real world is often broken down into pieces and stored across multiple tables.

Relating Tables

The most beneficial features of SQL and relational database systems is “Join” make relational database systems with “Relational”.

- Select data from multiple tables by matching values
- Relationship can be:

1-to-1

client -> mobile number

1-to-many

department id -> employee

many-to-1

university -> city

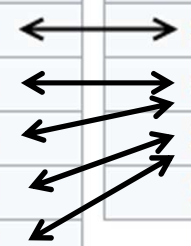
many-to-many

language -> country

Keys

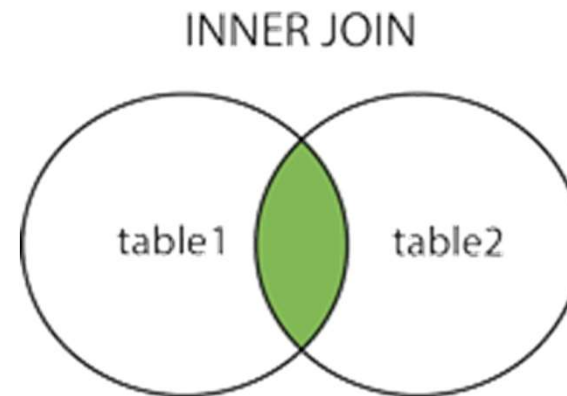
Every table should have a **primary key** that uniquely identifies each row.

Employee table		Department table	
LastName	DepartmentID	DepartmentID	DepartmentName
Rafferty	31	31	Sales
Jones	33	33	Engineering
Heisenberg	33	34	Clerical
Robinson	34	35	Marketing
Smith	34		
Williams	NULL		



Join Query

Employee table		Department table	
LastName	DepartmentID	DepartmentID	DepartmentName
Rafferty	31	31	Sales
Jones	33	33	Engineering
Heisenberg	33	34	Clerical
Robinson	34	35	Marketing
Smith	34		
Williams	NULL		



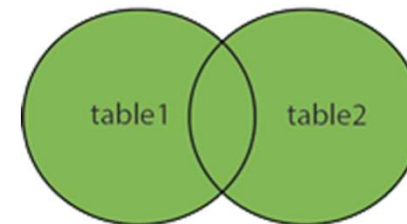
```
SELECT employee.LastName, employee.DepartmentID, department.DepartmentName
FROM employee
INNER JOIN department ON
employee.DepartmentID = department.DepartmentID
```

Employee.LastName	Employee.DepartmentID	Department.DepartmentName
Robinson	34	Clerical
Jones	33	Engineering
Smith	34	Clerical
Heisenberg	33	Engineering
Rafferty	31	Sales

Join Query

Employee table		Department table	
LastName	DepartmentID	DepartmentID	DepartmentName
Rafferty	31	31	Sales
Jones	33	33	Engineering
Heisenberg	33	34	Clerical
Robinson	34	35	Marketing
Smith	34		
Williams	NULL		

FULL OUTER JOIN



```
SELECT *
FROM employee FULL OUTER JOIN department
ON employee.DepartmentID = department.DepartmentID;
```

Employee.LastName	Employee.DepartmentID	Department.DepartmentName	Department.DepartmentID
Smith	34	Clerical	34
Jones	33	Engineering	33
Robinson	34	Clerical	34
Williams	NULL	NULL	NULL
Heisenberg	33	Engineering	33
Rafferty	31	Sales	31
NULL	NULL	Marketing	35

Exercise 2:

- Modify CUSTOMERS table as below:

customer_id (key)	name	last_name	district	moblie
20001	Andrew	LING	Mongkok	60123456
20002	Carol	CHAN	Tsuen wan	56780123
20003	Peter	PAN	Tue mun	52345678
20004	Daniel	YU	Causeway bay	91234567
20005	John	CHAN	Tue mun	90120001
20006	William	CHOY	North point	51232345
20007	John	LEE	Tsuen wan	62567890
20008	Carol	LAM	Kwun tong	96660001
20009	Mary	NG	Mongkok	98882345
20010	Mary	LEE	Tsuen wan	91191123

Exercise 2:

- Add PURCHASES table:

order_no (key)	customer_id	item_no	quantity	price
20210010	20001	10001	1	100
20210011	20002	10003	2	200
20210012	20003	10003	3	200
20210013	20004	10002	4	500
20210014	20005	10001	5	100
20210015	20006	10123	6	300
20210016	20007	10123	7	300
20210017	20008	10002	2	500
20210018	20009	10001	3	100
20210019	20010	10123	1	300
20210020	20010	10001	5	100

Exercise 2:

Question 1:

Write a query using a join to determine which items were ordered by each of the customers in the customers table. Select the customer_id, name, last_name, item_no, and price for everything each customer purchased in the purchases table.

Question 2:

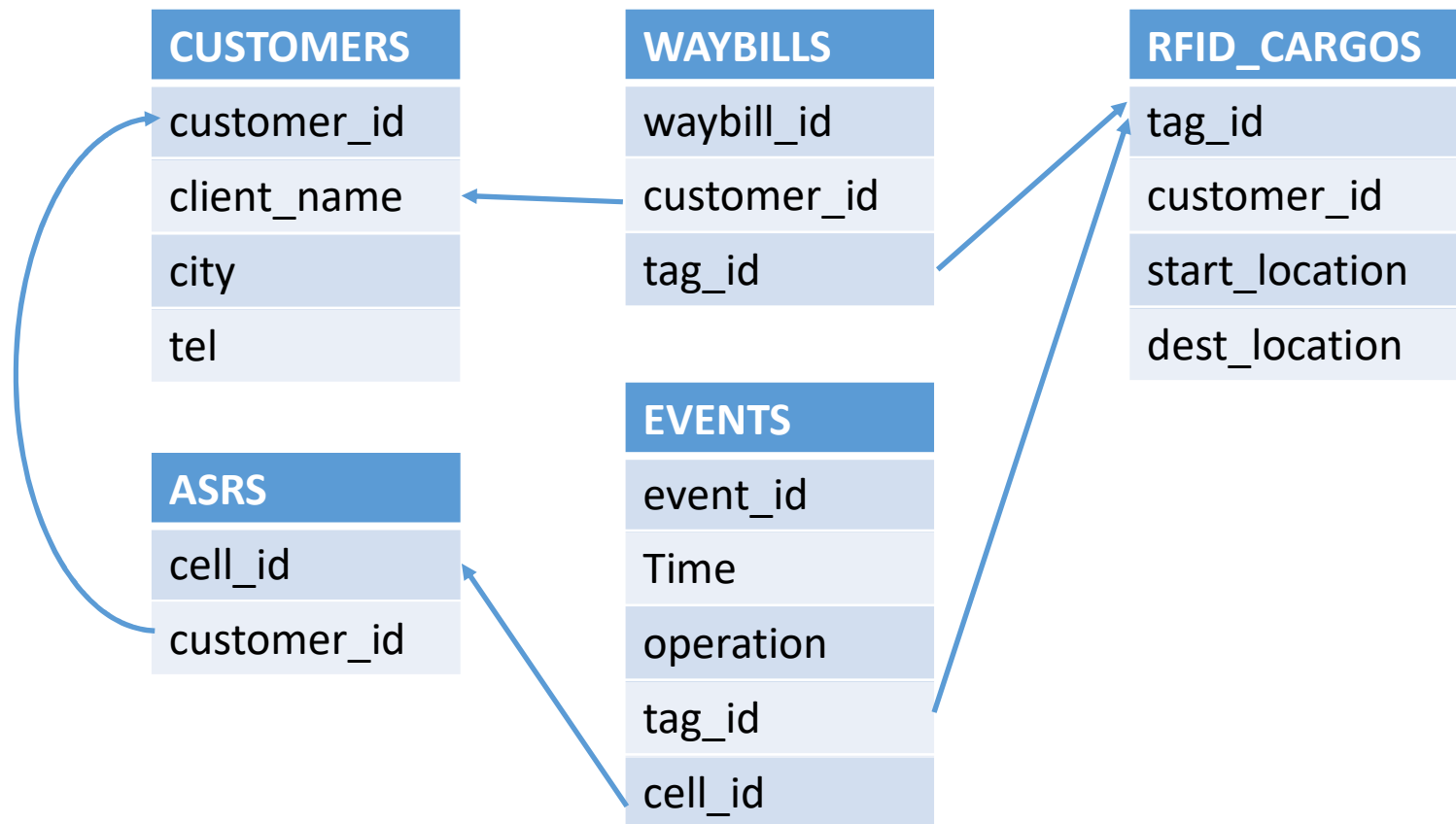
Repeat question 1, however display the results sorted by district in descending order.

Other Multiple Tables Example

CUSTOMERS

<u>customer_id</u>	<u>client_name</u>	city	<u>tel</u>
1	Andrew	HONG KONG	12345678
2	Daniel	TOKYO	23456789
3	David	NEWYORK	34567890
4	John	TAIPEI	45678901
5	Noah	BEIJIN	56789012
6	William	SHANGHAI	67890123

ER Diagram of system



CUSTOMERS Table

CUSTOMERS

customer_id

client_name

address

tel

customer_id	client_name	city	tel
1	Andrew	HONG KONG	12345678
2	Daniel	TOKYO	23456789
3	David	NEWYORK	34567890
4	John	TAIPEI	45678901
5	Noah	BEIJIN	56789012
6	William	SHANGHAI	67890123

WAYBILLS Table

WAYBILLS

waybill_no

customer_id

tag_id

waybill_no	customer_id	tag_id
1001	1	10000
1002	2	20000
1003	3	30000
1004	4	40000
1005	5	50000
1006	6	60000

RFID_CARGOS Table

RFID_CARGOS

tag_id

customer_id

start_location

dest_location

tag_id	customer_id	start_location	dest_location
123456	1	HONG KONG	BEIJIN
234567	2	TOKYO	HONG KONG
345678	3	NEWYORK	TOKYO
456789	4	BEJIN	TAIPEI
567890	5	TAIPEI	NEWYORK
678901	6	HONG KONG	SHANGHAI

ASRS Table

ASRS

cell_id

customer_id

cell_id	customer_id
101	1
102	2
103	3
104	4
105	5
106	6

EVENTS Table

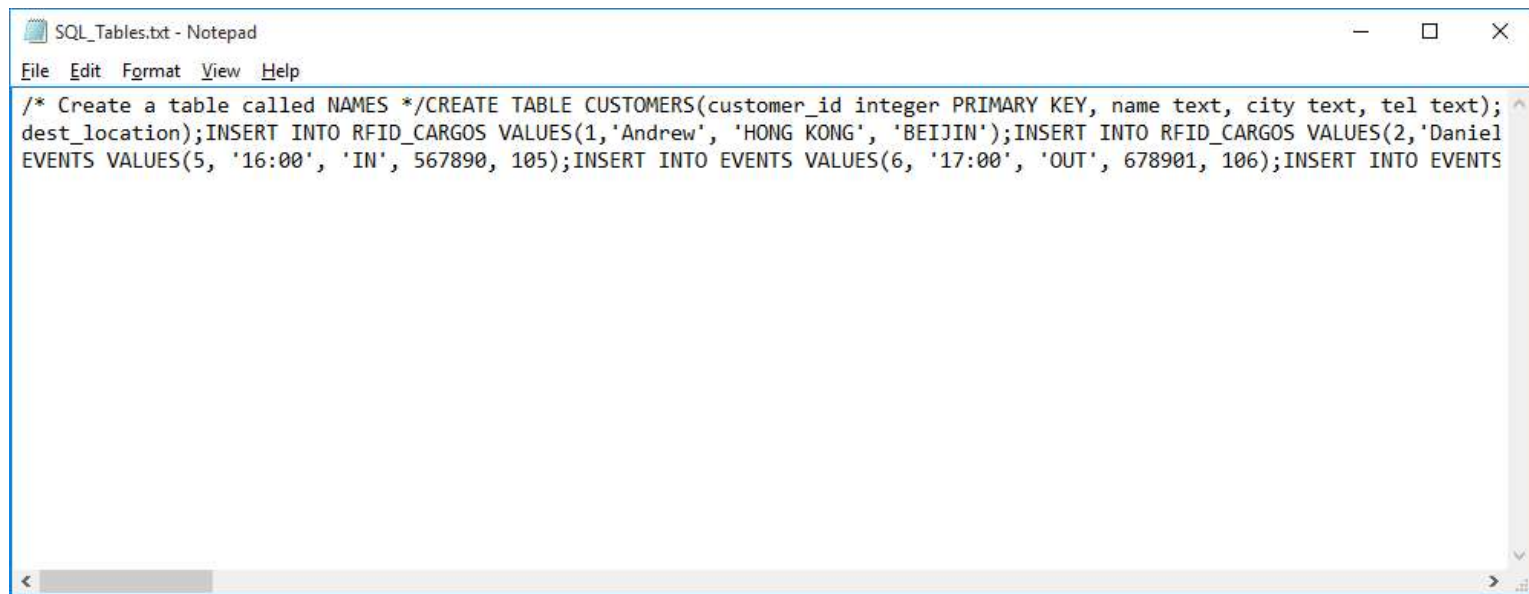
EVENTS
event_id
Time
operation
tag_id
cell_id

event_id	time	operation	tag_id	Cell_id
1	12:00	IN	123456	101
2	13:00	IN	234567	102
3	14:00	OUT	345678	103
4	15:00	IN	456789	104
5	16:00	IN	567890	105
6	17:00	OUT	678901	106

Optional SQL exercises

1. Find how many “IN” events in EVENTS table.
2. Find how many events occurred after 16:00 in EVENTS table.
3. Given event_id=1, find the cell_id from EVENTS table.
4. Write a SQL statement with subquery that find query client name from EVENTS with event_id = 2.

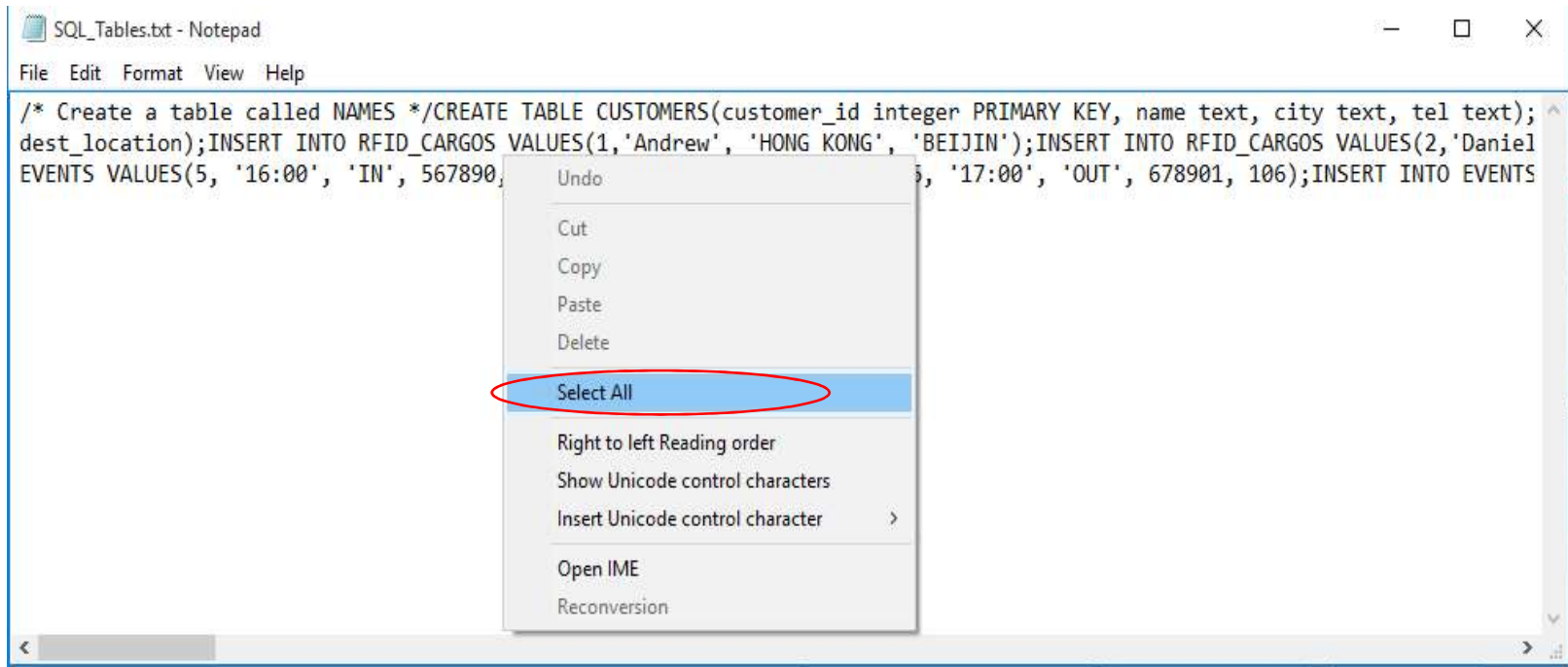
Open database file and select all data



```
SQL_Tables.txt - Notepad
File Edit Format View Help
/* Create a table called NAMES */CREATE TABLE CUSTOMERS(customer_id integer PRIMARY KEY, name text, city text, tel text);
dest_location);INSERT INTO RFID_CARGOS VALUES(1,'Andrew', 'HONG KONG', 'BEIJIN');INSERT INTO RFID_CARGOS VALUES(2,'Daniel
EVENTS VALUES(5, '16:00', 'IN', 567890, 105);INSERT INTO EVENTS VALUES(6, '17:00', 'OUT', 678901, 106);INSERT INTO EVENTS
```

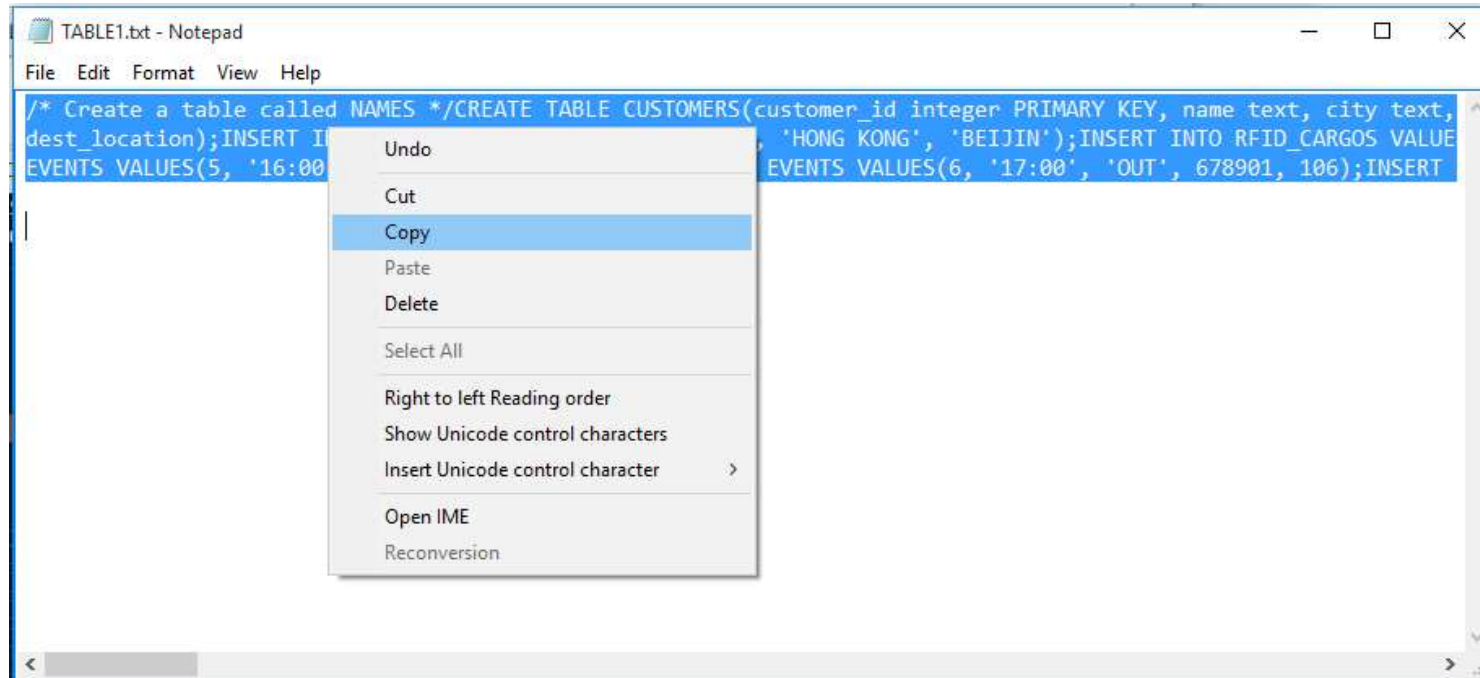
Open the file “SQL_Tables.txt”(given by tutor) by “Notepad”.

Select All from text file



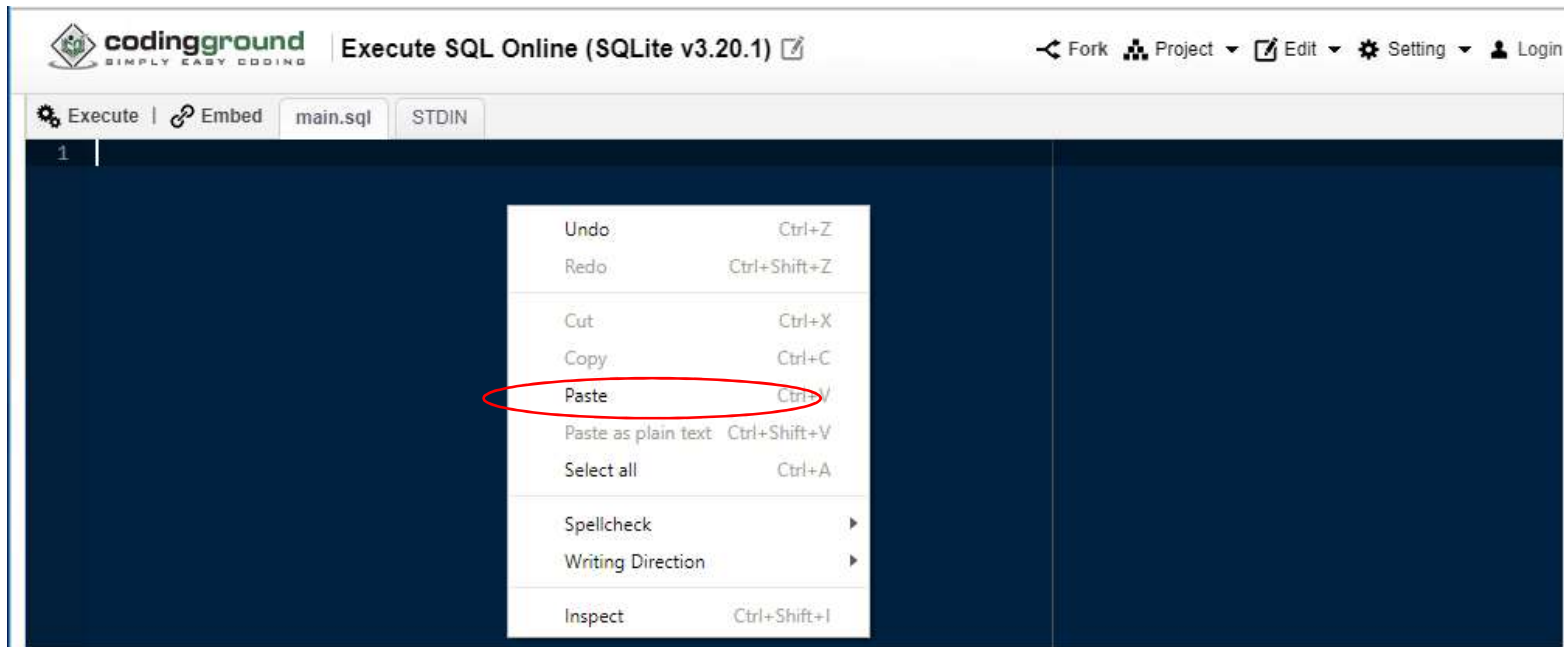
Right-click on Notepad's screen then click "Select All" or press "Ctrl-A" key to copy all text.

Copy database form text file



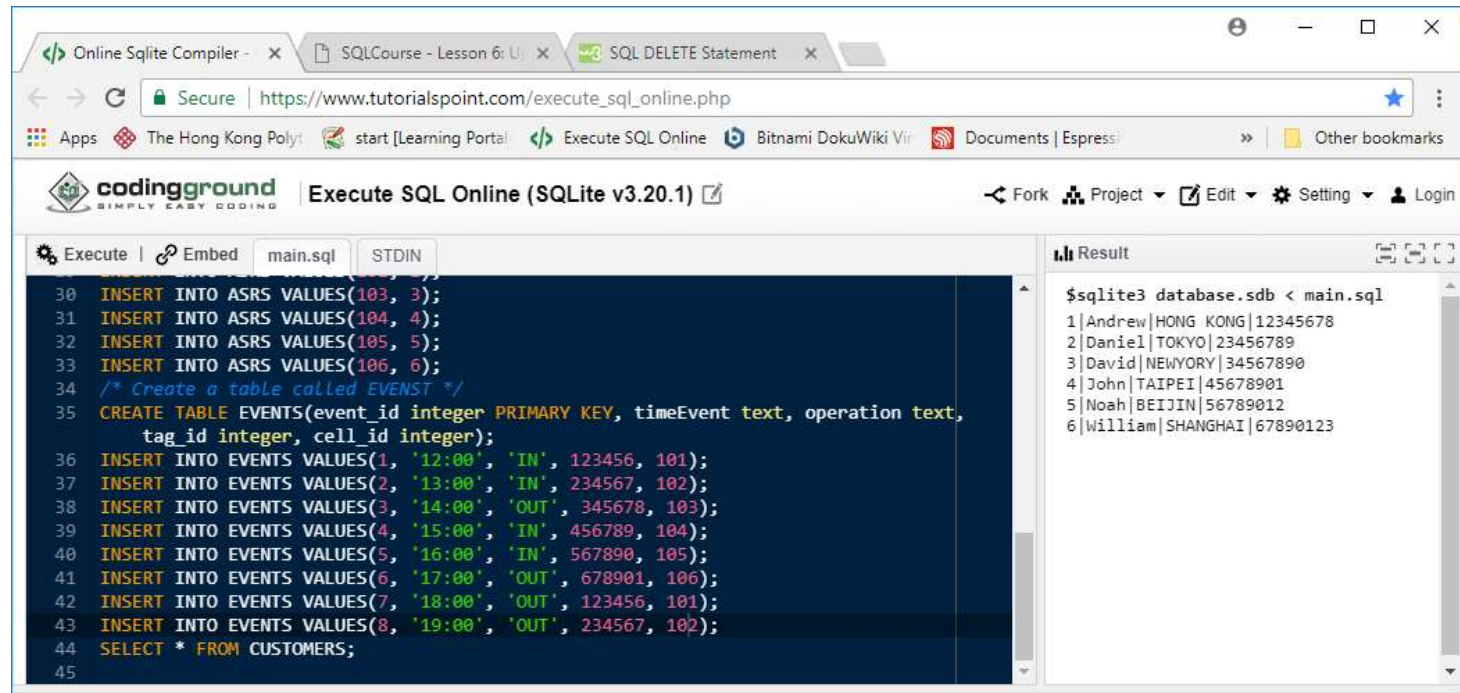
Right-click on Notepad's screen then click "Copy".

Paste database to SQL Online



Click on the “Execute SQL Online” screen to paste the database.

Execute SQL statement



The screenshot shows a web browser window with the URL https://www.tutorialspoint.com/execute_sql_online.php. The page title is "Execute SQL Online (SQLite v3.20.1)". The interface includes a "Execute" button and a "Result" panel. The SQL code in the editor is as follows:

```
30 INSERT INTO ASRS VALUES(103, 3);
31 INSERT INTO ASRS VALUES(104, 4);
32 INSERT INTO ASRS VALUES(105, 5);
33 INSERT INTO ASRS VALUES(106, 6);
34 /* Create a table called EVENST */
35 CREATE TABLE EVENTS(event_id integer PRIMARY KEY, timeEvent text, operation text,
    tag_id integer, cell_id integer);
36 INSERT INTO EVENTS VALUES(1, '12:00', 'IN', 123456, 101);
37 INSERT INTO EVENTS VALUES(2, '13:00', 'IN', 234567, 102);
38 INSERT INTO EVENTS VALUES(3, '14:00', 'OUT', 345678, 103);
39 INSERT INTO EVENTS VALUES(4, '15:00', 'IN', 456789, 104);
40 INSERT INTO EVENTS VALUES(5, '16:00', 'IN', 567890, 105);
41 INSERT INTO EVENTS VALUES(6, '17:00', 'OUT', 678901, 106);
42 INSERT INTO EVENTS VALUES(7, '18:00', 'OUT', 123456, 101);
43 INSERT INTO EVENTS VALUES(8, '19:00', 'OUT', 234567, 102);
44 SELECT * FROM CUSTOMERS;
45
```

The "Result" panel displays the output of the SQL statements:

```
$sqlite3 database.sdb < main.sql
1|Andrew|HONG KONG|12345678
2|Daniel|TOKYO|23456789
3|David|NEWYORK|34567890
4|John|TAIPEI|45678901
5|Noah|BEIJIN|56789012
6|William|SHANGHAI|67890123
```

1. Enter SQL statement "SELECT * FROM CUSTOMERS;"
2. Click on  Execute .
3. Result show on "Result" screen.